```
import ai_thinking
from future import possibilities

def start_journey():
  return "Knowledge"
```

<CHAPTER_03 />

PYTHON FOUNDATIONS FOR AI

AI Thinking: A Hands-On Introduction

```
System.init(Chapter_3)
Loading modules...
[OK] Variables
[OK] Loops
[OK] Logic
```

MISSION PARAMETERS

STATUS: INITIALIZING
MODULES: 5
TARGET: MASTERY

CHAPTER 3 LEARNING OBJECTIVES

01 Write Python code using **variables, data types, and operators**

02 Create and manipulate **lists, dictionaries, and collections**

03 Implement control flow using **conditionals and loops**

04 Define and call **functions** with parameters

05 Import and use external **Python libraries**

WHY PYTHON MATTERS

READABLE SYNTAX

Python reads like executable pseudocode. It removes the barrier between **thinking** and **coding.**

MASSIVE ECOSYSTEM

The world's best AI tools (PyTorch, TensorFlow, Scikit-learn) are all built **native to Python.**

INDUSTRY STANDARD

From Google DeepMind to OpenAI, Python is the **lingua franca** of modern AI research.

PYTHON CORE

The Foundation

AI LIBRARIES

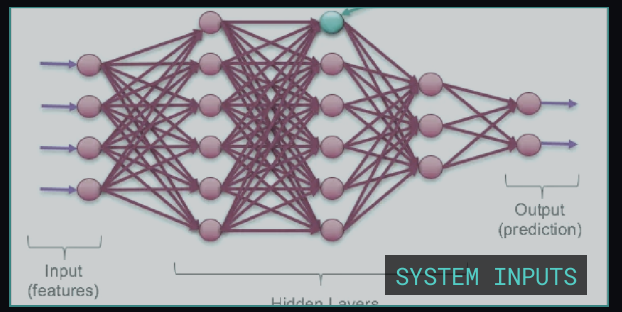
Pandas, NumPy, PyTorch

AI APPLICATIONS

ChatGPT, Tesla Autopilot, Netflix Recs

VARIABLES & DATA TYPES

A **variable** is a named container that stores a value. In Python, every value has a specific **type** that determines what you can do with it.



int

Whole numbers without decimals. Used for counts, years, indices.

```
age = 21
year = 2024
```

float

Numbers with decimal points. Used for precision, prices, science.

```
temp = 98.6
pi = 3.14
```

str

Text enclosed in quotes. Used for names, messages, data.

```
name = "AI"
id = '007'
```

bool

True or False values. Used for logic and decisions.

```
active = True
done = False
```

None

Represents the absence of a value. A placeholder.

```
data = None
empty = None
```

LISTS: ORDERED COLLECTIONS [ ]

// DEFINITION

A **List** is a mutable, ordered sequence of items. It can hold different data types and allows duplicates.

```
data = ["AI", 42, 3.14, True]
```

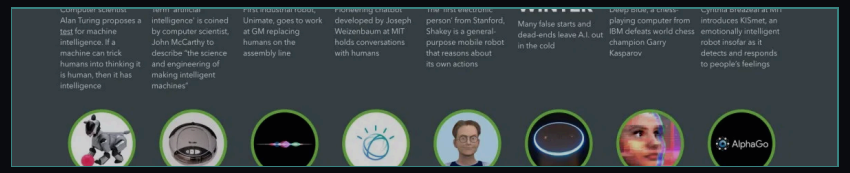
0	1	2
"AI"	42	3.14
-4	-3	-2

ZERO-BASED INDEXING

```
my_list = ["AI", 42, 3.14]
# Access
first = my_list[0]   # "AI"
last = my_list[-1]   # 3.14
# Modify
my_list[1] = 100
```

- Ordered Sequence
- Mutable (Changeable)
- Mixed Data Types

// REAL WORLD ANALOGY



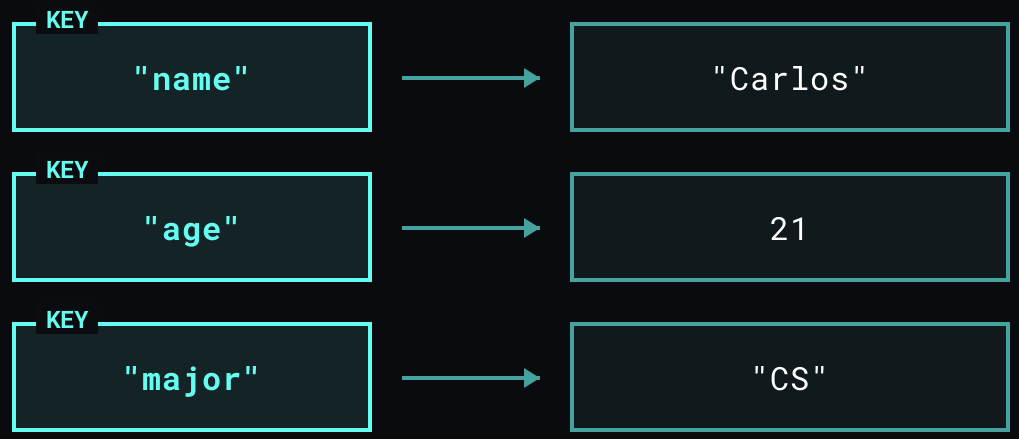
Think of a list like a **Timeline.** Events are stored in a specific order. You can find an event by its position or add new events to the end.

DICTIONARIES: KEY-VALUE PAIRS

student_data.py

```python
student = {
    "name": "Carlos",
    "age": 21,
    "major": "CS"
}

# Access by Key
print(student["name"])
```



Unique Keys

Unordered Collection

Fast Lookup

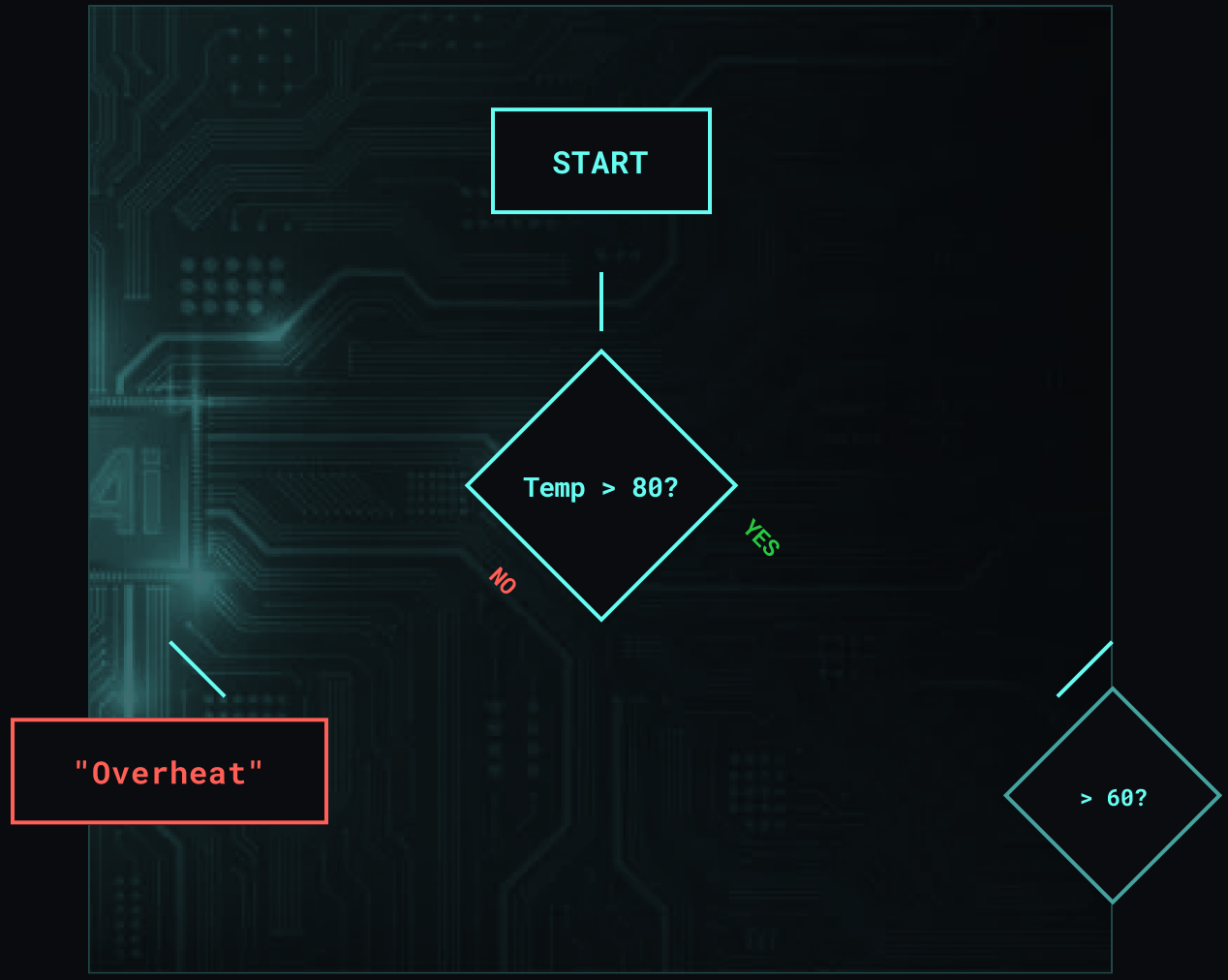
Mutable (Changeable)

MAKING DECISIONS: CONDITIONALS

logic_gate.py

```
# Check system status
temp = 85

if temp > 80:
    print("Warning: Overheat")
elif temp > 60:
    print("Status: Normal")
else:
    print("Status: Cold")
```



REPETITION PROTOCOLS

SYSTEM.LOOP(TRUE)

FOR LOOP

Iterates over a **known sequence** (like a list or range). Best for processing data collections.

PYTHON

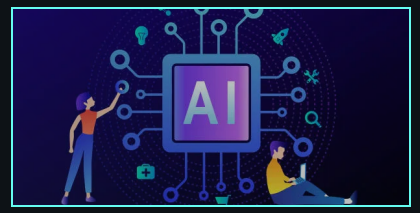
```python
for item in data_list: # Process each item
print(f"Processing {item}")
```

WHILE LOOP

Repeats **while a condition is true.** Best for indefinite tasks or waiting for input.

PYTHON

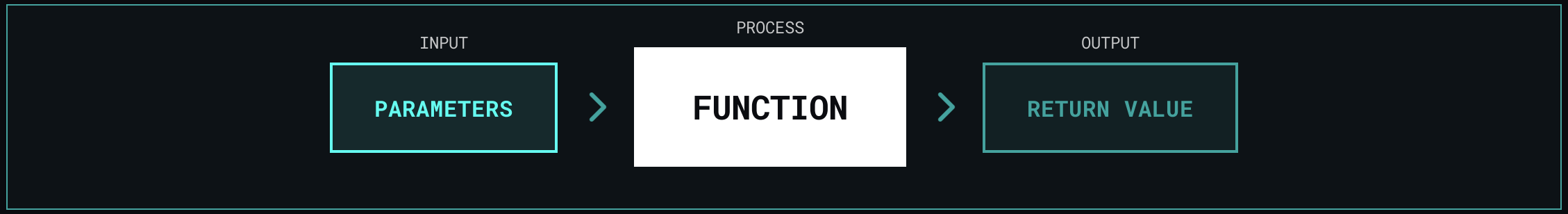
```python
active = True while active: # Run until
stopped data = sensor.read()
```



> AUTOMATION STATUS: ACTIVE
> PROCESSING DATA STREAMS...
> OPTIMIZING REPETITIVE TASKS...

Loops allow AI systems to process millions of data points efficiently.

FUNCTIONS: REUSABLE MODULES




```python
def calculate_total(price, tax):
    # Process data
    total = price + (price * tax)
    return total

# Call the module
result = calculate_total(100, 0.07)
```

MODULARITY

Break complex problems into small, manageable pieces.

REUSABILITY

Write code once, use it everywhere. Don't repeat yourself (DRY).

READABILITY

Named functions make code read like English instructions.

SYS.MOD.LOADED
MEM_ADDR: 0x4F3A

LIBRARIES: THE AI TOOLKIT

// DON'T REINVENT THE WHEEL. IMPORT IT.

main.py

```
import math
import random
import numpy as np   # Standard alias
import pandas as pd  # Standard alias
```

PYTHON STANDARD LIBRARY

math

Basic mathematical functions (sqrt, sin, cos, pi). Built-in and ready to use.

random

Generate random numbers and make random choices. Essential for simulations.

DATA SCIENCE CORE

NumPy

Numerical Python. High-performance arrays and matrix math. The foundation of all AI libraries.

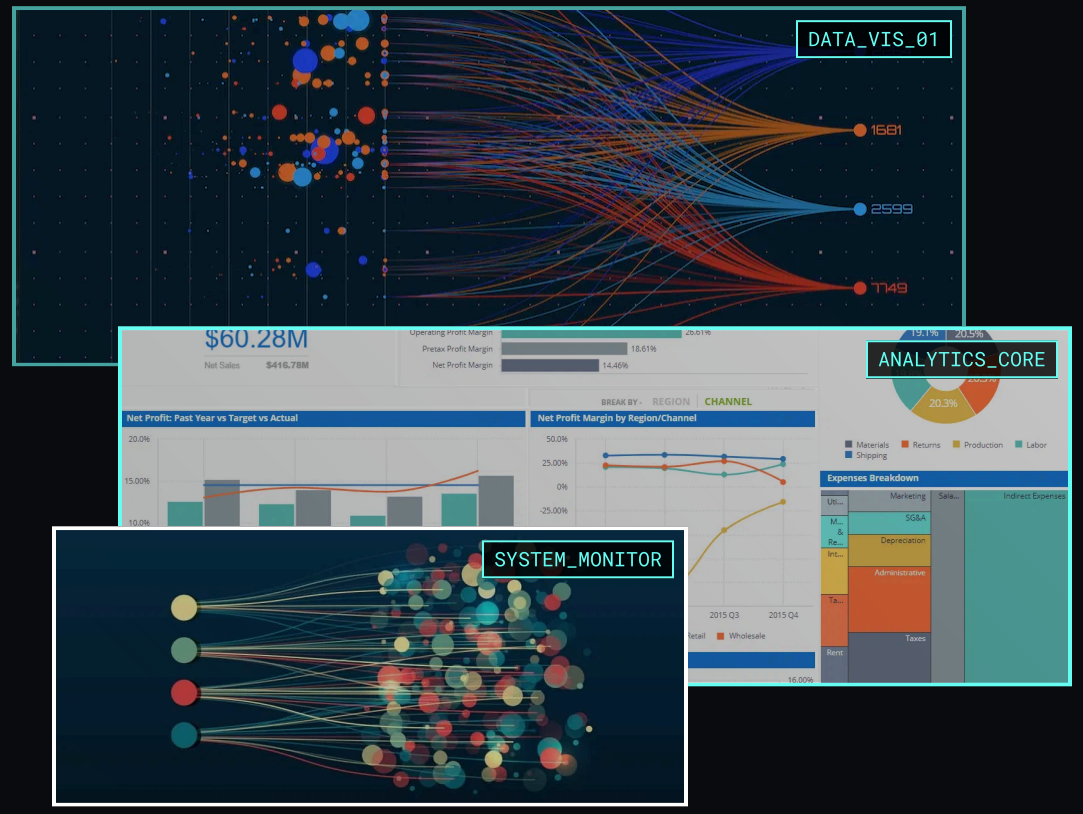
pandas

Python Data Analysis. Powerful data structures (DataFrames) for manipulating structured data.

MODULES_LOADED: 4
SYSTEM_READY: TRUE

MISSION DEBRIEF: PYTHON CORE

- **VARIABLES & TYPES**
  Storing data in labeled containers
- **COLLECTIONS**
  Organizing data in Lists and Dictionaries
- **CONTROL FLOW**
  Directing logic with If/Else and Loops
- **FUNCTIONS**
  Creating reusable, modular code blocks
- **LIBRARIES**
  Leveraging the Python ecosystem (NumPy, Pandas)



> SYSTEM STATUS: READY

> NEXT MODULE: DATA ANALYSIS

NEXT UP: DATA ANALYSIS

APPLYING PYTHON TO THE REAL WORLD

- Loading Real Datasets
- Cleaning Messy Data
- Visualizing Insights

chapter_04_preview.py

```python
import pandas as pd

# 1. Load Data
df = pd.read_csv("sales_data.csv")

# 2. Analyze
print(df.describe())

# 3. Visualize
df.plot(kind="bar")
```